550. Game Play Analysis IV

Medium | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Activity`

```
+--------------+---------+
| Column Name  | Type    |
+--------------+---------+
| player_id    | int     |
| device_id    | int     |
| event_date   | date    |
| games_played | int     |
+--------------+---------+
(player_id, event_date) is the primary key (combination of columns with unique values) of this table.
This table shows the activity of players of some games.
Each row is a record of a player who logged in and played a number of games (possibly 0) before logging out
on someday using some device.
```

Write a solution to report the **fraction** of players that logged in again on the day after the day they first logged in, **rounded to 2 decimal places**. In other words, you need to determine the number of players who logged in on the day immediately following their initial login, and divide it by the number of total players.

The result format is in the following example.

Example 1:

Input:

Activity table:

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-03-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

Output:

fraction
0.33

Explanation:

Only the player with id 1 logged back in after the first day he had logged in so the answer is 1/3 = 0.33

SQL function	Purpose
`MIN()`	Finds the earliest login date
`GROUP BY`	Groups records player-wise
`DATE_ADD()`	Adds one day to a date
`JOIN`	Matches a player's expected date with an actual login
`COUNT()`	Counts qualifying players
`ROUND()`	Rounds the fraction to 2 decimal places

PS / Day 10 / 7-11-26.

leetcode-550, Game Play Analysis IV.

imagine we have a gaming application. whenever a player logs in & plays a game, one record is inserted into the Activity table.
we have 4 columns:-

Column	Meaning
player-id	unique ID of the player
device-id	device used by " ".
event-date	date on which the player logged in.
game-played	no. of games played.

Our task:
find the fraction of players who logged in again exactly one day after their FIRST login.
formula =
Fraction = players who returned the next day / total distinct players
finally, round the answer to 2 decimal places.

example ⇒

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-03-02	6
1	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5.

we have two distinct players:-
- player 1
- player 3.

player 1 ⇒ 1st login : 2016-03-01
next day login : 2016-03-02.
qualify ⇒ player 1 returned the very next day.

player 3 ⇒ 1st login : 2016-03-02
→ expected next-day login : 2016-03-03
→ actual next login : 2018-07-03.
distinct qualify ⇒ 0

therefore :- 1/2 = 0.50.

We need to perform 3 operations:
1- find the 1st login date of every player.
2- check whether that player has another record on the very next day.
3- divide the number of qualifying players by the total no. of players.

Step 1 ⇒ find the 1st login date of each player.

```
SELECT
    player_id,
    MIN(event_date) AS first_login
FROM Activity
GROUP BY player_id;
```

→ finds the earliest login date for each player.

complete code ✓

```
SELECT
  ROUND(
    COUNT(DISTINCT a.player_id) * 1.0 /
    (SELECT COUNT(DISTINCT player_id) FROM Activity),
  2) AS fraction
FROM Activity AS a
JOIN
    (SELECT
        player_id,
        MIN(event_date) AS first_login
     FROM Activity
     GROUP BY player_id) AS f
ON a.player_id = f.player_id
AND a.event_date = DATE_ADD(f.first_login, INTERVAL 1 DAY);
```

SELECT → we specify what we want to display.

ROUND(→ rounds the result to 2 decimal places, e.g. ROUND(0.3333, 2) = 0.33.

COUNT(DISTINCT a.player_id) → removes duplicate player IDs → represents players who logged in the next day.

* 1.0 → ensures decimal division, prevents int. division.

(SELECT COUNT(DISTINCT player_id) FROM Activity) → counts all unique players. This is our denominator.

2 → means the result to 2 decimal place.

) AS fraction → gives our o/p column the name fraction.

FROM Activity AS a

JOIN → checks whether the expected next-day login actually exists

(SELECT player_id, MIN(event_date) AS first_login FROM Activity GROUP BY player_id) → groups all records belonging to the same player. finds the earliest date separately for each player.

AS f → give the entire subquery an alias f.

ON a.player_id = f.player_id

AND a.event_date = DATE_ADD(f.first_login, INTERVAL 1 DAY); → calculate the exact next calender day.

Last 2nd Lene

↳ matches the same player id to the records.

↳ makes sure we compare a player's login with their own 1st login date.

Last Lene → takes the player's 1st login date
→ adds exactly one day
→ checks whether the actual login date matches the calculated next-day date.

Accepted 15 / 15 testcases passed

Akankshyaa submitted at Nov 07, 2026 08:37

Solution

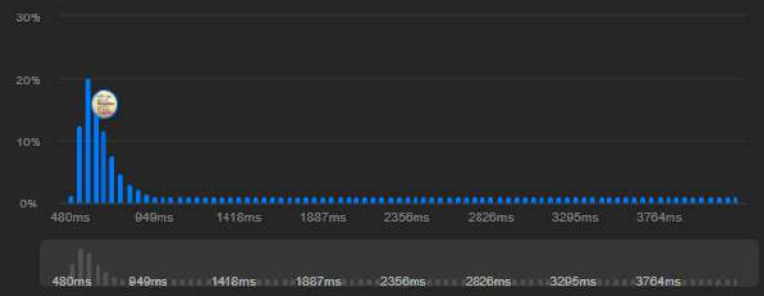
Save 41%

Big Diwali Dhamaka

Unlock the Full Experience with Premium for ₹583.25/mo — Ends Nov 8!

Runtime

702 ms | Beats **34.96%**



Code | MySQL

```mysql
SELECT
ROUND(
    COUNT(DISTINCT a.player_id)*1.0/
    (SELECT COUNT(DISTINCT player_id)
    FROM Activity),2)
    AS fraction
    FROM Activity AS a
    JOIN(
```

View more

```mysql
SELECT
ROUND(
    COUNT(DISTINCT a.player_id)*1.0/
    (SELECT COUNT(DISTINCT player_id)
    FROM Activity),2)
    AS fraction
    FROM Activity AS a
    JOIN(
    SELECT
    player_id,
    MIN(event_date) AS first_login
FROM Activity
GROUP BY player_id)
AS f
ON a.player_id=f.player_id
AND a.event_date= DATE_ADD(f.first_login,INTERVAL 1 DAY);
```

Saved

Testcase | Test Result

Accepted Runtime: 91 ms

Case 1

Input

Activity =

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-03-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

Output

| fraction |